

UNIDADE 4

CONTROL DE EXCEPCIÓNS

v1.3

PROGRAMACIÓN

CICLO SUPERIOR DE DAM

Manuel Pacior Pérez

Índice

1 INTRODUCCIÓN.....	3
1.1 Erros.....	3
1.1.1 Erros de Compilación (Compilation Error):.....	3
1.1.2 Erros de Tempo de Execución (Runtime Error).....	3
1.2 Excepcións.....	4
1.3 Erros vs excepcións.....	4
2 O BLOQUE TRY-CATCH-FINALLY.....	5
2.1 Estrutura básica.....	5
2.2 Boas prácticas.....	6
2.2.1 Exemplo.....	6
3 A INSTRUCCIÓN THROW.....	7
4 TIPOS DE EXCEPCIÓNS.....	7
4.1 Excepcións comprobadas (Checked Exceptions).....	8
4.1.1 Características principais.....	8
4.1.2 Exemplos de excepcións comprobadas.....	8
4.1.3 Boas prácticas no uso de excepcións comprobadas.....	9
4.2 Excepcións non comprobadas (Unchecked Exceptions).....	10
4.2.1 Características principais.....	10
4.2.2 Exemplos de excepcións non comprobadas.....	10
4.2.3 Boas prácticas no uso de excepcións non comprobadas.....	11
5 PROPAGACIÓN DE EXCEPCIÓNS: INSTRUCCIÓN THROWS.....	12



1 Introducción

En Java, as excepcións e os erros son mecanismos fundamentais que permiten tratar situacións inesperadas ou problemas durante a execución dun programa. Estes mecanismos axudan a escribir código robusto e a manexar situacións excepcionais de maneira controlada. Vamos explorar detalladamente os conceptos de excepcións e erros en Java.

1.1 Erros

Os erros en Java son situacións graves que xeralmente indican problemas que non se poden manexar facilmente durante a execución normal do programa. Trátanse de subtipos da clase [*Error*](#) e poden interromper o fluxo normal do programa.

Os erros graves que derivan de [*Error*](#) adoitan ser xestionados por unha combinación de probas e accións correctivas durante o desenvolvemento e non se espera que o código normal da aplicación manexe os erros.

Podemos ter dous tipos de erros, en tempo de compilación ou en tempo de execución.

1.1.1 Erros de Compilación (Compilation Error):

Son erros detectados polo compilador durante a fase de compilación, e implican que o programa non se poderá executar ata que se corrixa. Como saltan antes da execución da aplicación, non son tan críticos. Algúns exemplos que poden facer saltar este tipo de erros inclúen: sintaxe incorrecta, declaración de variables non válidas, asignación de tipos incorrectos, etc.

1.1.2 Erros de Tempo de Execución (Runtime Error)

Son erros que ocorren durante a execución do programa e poden levar á súa terminación abrupta. Algúns exemplos disto serían os seguintes:

- **OutOfMemoryError:** Ocorre cando a JVM queda sen memoria suficiente para colocar en memoria novos obxectos.
- **StackOverflowError:** Pode suceder cando hai demasiadas chamadas recursivas a un método sen que exista unha condición de saída.
- **NoClassDefFoundError:** Pode darse cando a JVM non pode atopar a definición dunha clase que foi usada durante a compilación.
- **UnsatisfiedLinkError:** Ocorre cando unha biblioteca nativa necesaria non se pode cargar.

1.2 Excepcións

Unha excepción en Java é un evento que ocorre durante a execución dun programa e que interrompe o seu fluxo normal. Pode deberse a erros lóxicos, condicións imprevistas ou situacións anómalas. Trátanse de subtipos da clase [*Exception*](#) e pola contra do que acontece no caso dos erros, neste caso representa problemas que non son críticos, e que se poden (e deben) tratar.

Nesta unidade imos centrarnos no estudo dos diferentes tipos de excepcións en Java, e como se deben manexar.

1.3 Erros vs excepcións

No seguinte recadro podemos ver un pequeno resumo das diferenzas entre erros e excepcións:

ASPECTO	EXCEPCIÓNS	ERROS
Definición	Eventos excepcionais que ocorren en tempo de execución e que poden ser tratados ou recuperados.	Problemas graves xeralmente relacionados co sistema ou coa JVM que non se poden recuperar.
Clase Base	Derivan da clase <code>java.lang.Exception</code> .	Derivan da clase <code>java.lang.Error</code> .
Gravidade	Menor: Representan problemas previsibles no programa.	Maior: Indican problemas críticos ou irreparables.
Manexo	Pódense tratar utilizando bloques try-catch ou declarar con throws.	Non se poden tratar directamente polo programa.
Causas Comúns	- Fallos en operacións aritméticas. - Acceso indebido a arrays. - Conversións de tipo ou formato inválidos. - Acceso a obxectos null.	- Falta de memoria. - Problemas na recursividade - Fallos na carga de clases ou bibliotecas.
Recuperación	Normalmente recuperable se se manexan axeitadamente.	Xeralmente non recuperable; require a intervención do sistema ou axustes na configuración.
Exemplo Común	<code>NullPointerException</code> , <code>ArithmeticException</code> , <code>ArrayIndexOutOfBoundsException</code> .	<code>OutOfMemoryError</code> , <code>StackOverflowError</code> , <code>NoClassDefFoundError</code> .
Uso Habitual	Relacionadas coas regras de negocio e operacións do programa.	Relacionadas co estado do sistema ou o funcionamento da JVM.
Impacto na JVM	Non afecta o funcionamento global da JVM.	Pode deter ou colapsar a JVM.
Control Obrigatorio	As excepcións controladas deben ser tratadas obrigatoriamente (compilador forza a tratar ou declarar).	Non é obrigatorio nin posible manexalas no código; considéranse imprevísibles.

	As non controladas son opcionais.	
Subclasificación	- Controladas: IOException, SQLException (revisadas polo compilador). - Non controladas: NullPointerException, ArithmeticException (non revisadas).	Non hai subclasificación formal, pero exemplos típicos inclúen erros de memoria ou recursividade excesiva.
Cando Usar	Para capturar e tratar problemas esperados en tempo de execución.	Para problemas críticos que requiren axustes externos ou revisión do código/xestión de memoria.

2 O bloque *try-catch-finally*

O bloque *try-catch-finally* en Java é unha construción que permite manexar excepcións, e está deseñado para envolver un conxunto de declaracións, que poden lanzar diferentes excepcións, permitindo a súa captura e xestión.

2.1 Estrutura básica

A estrutura básica dun bloque *try-catch-finally* é a seguinte:

```
try {
    // Código que pode lanzar excepcións
} catch (TipoDeExcepcion e) {
    // Xestión da excepción
} finally {
    // Este bloque sempre se executará ao final, haxa ou non unha excepción
}
```

A continuación imos realizar unha descrición detallada dos diferentes bloques que define cada unha das instrucións:

- ✓ **try**: Todas as declaracións no bloque try son executadas. Se se produce unha excepción, a execución do bloque try interrompese e o control pasa ao bloque catch.
- ✓ **catch**:
 - Se ocorre unha excepción no bloque try, o control pasa ao primeiro bloque catch cuxo tipo coincide co tipo da excepción lanzada.
 - Pode haber varios bloques catch para manexar diferentes tipos de excepcións.

```
try {
    // Código que pode lanzar excepcións
} catch (ExceptionTipo1 e1) {
    // Xestión para ExceptionTipo1
}
```

```
} catch (ExceptionTipo2 e2) {  
    // Xestión para ExceptionTipo2  
}  
catch (Exception e) {  
    // Xestión xeral para outras excepcións  
}
```

✓ **Finally:**

- O bloque `finally` é opcional e proporciona unha sección de código que se executa sempre, independentemente de se se lanza ou captura unha excepción.
- Utilízase para realizar limpezas ou liberación de recursos, asegurándose de que se executa ao final, mesmo cando se produciu un erro.

2.2 Boas prácticas

Cando traballamos con excepcións usando o bloque *try-catch-finally* debemos ter en conta as seguintes recomendacións:

- ➔ **Capturar excepcións específicas:** Capturar excepcións específicas antes de excepcións máis xerais permite un manexo máis preciso.
- ➔ **Manexo de recursos:** O bloque `finally` é útil para liberar recursos, como peche de ficheiros ou conexións a bases de datos, asegurándose de que se realice independentemente de excepcións.

2.2.1 Exemplo

A continuación amosamos un exemplo práctico de uso do bloque *try-catch-finally*:

```
try {  
    // Código que pode lanzar excepcións  
    int resultado = dividir(val1, val2);  
} catch (ArithmeticException e) {  
    // Xestión para división por cero, no caso de que val2 sexa 0  
    System.out.println("Non se pode dividir por cero.");  
} finally {  
    // Código que se executa sempre, independentemente de se se lanzou ou  
    // capturou unha excepción  
    System.out.println("Finalizando a execución.");  
}
```

Neste exemplo, se a función `dividir` lanza unha excepción de *ArithmeticException* (división por cero), o bloque `catch` correspondente que capture ese tipo de excepción, executarase interrompendo o fluxo de execución do `try`. Finalmente executarase o bloque `finally`. Este

último asegura que se realice calquera limpeza necesaria ou liberación de recursos, independentemente de se ocorreu un erro ou non.

3 A instrución *throw*

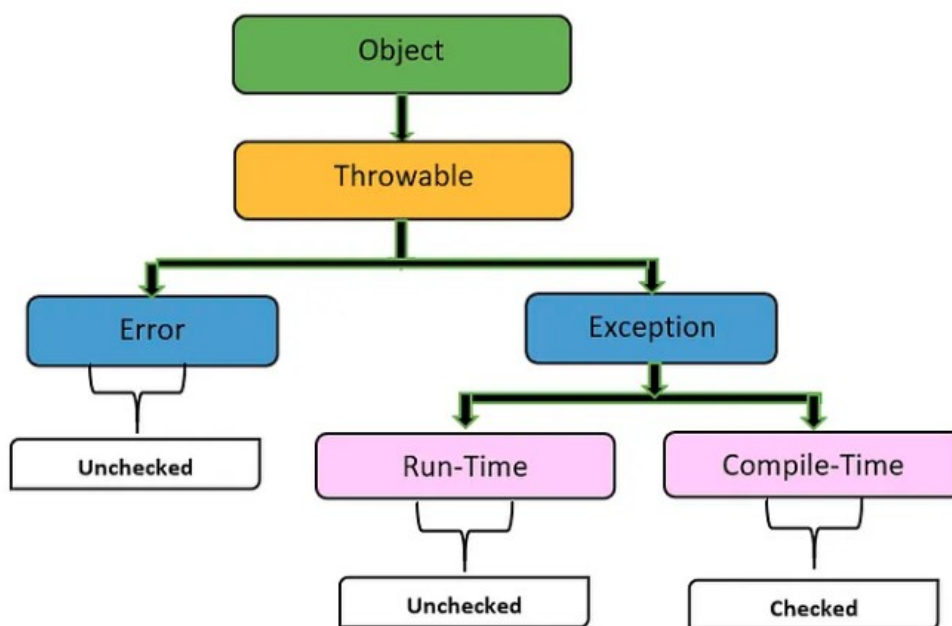
Está instrución utilízase para lanzar unha excepción de xeito manual. Nesta unidade abordaremos que son e para que serven. Fíxate que as excepcións que nós trataremos na nosa aplicación son todas instancias de tipo *Exception* (ou clases estendidas, concepto que veremos máis adiante), e que cando queremos lanzar un evento deste tipo, facémolo coa instrución *throw* seguido da instrución *new* (creamos unha instancia no mesmo momento) co tipo de excepción.

Exemplo:

```
if (condición) {  
    throw new Exception("Mensaxe de erro"); // Lanza unha excepción xenérica  
}
```

4 Tipos de excepcións

Existen dous tipos principais de excepcións en Java: excepcións comprobadas (checked exceptions) e excepcións non comprobadas (unchecked exceptions).



Imaxe. Xerarquía dos erros e excepcións (obtido da [páxina web](#))

4.1 Excepcións comprobadas (Checked Exceptions)

As excepcións comprobadas en Java requiren que o programador as manexe de maneira explícita mediante cláusulas *try-catch* ou declarándoas na sinatura do método. Son excepcións que o compilador obriga a ser tratadas ou propagadas.

Son, polo tanto, todas aquelas excepcións que o compilador obriga a xestionar mediante o uso de *try-catch* ou declarándoas na sinatura do método.

Exemplo:

```
try {  
    // Código que pode lanzar unha excepción comprobada  
} catch (TipoDeExcepcion e) {  
    // Xestionar a excepción  
}
```

4.1.1 Características principais

A continuación, exploraremos as características e algúns exemplos de excepcións comprobadas:

- ✓ **Requiren throws ou try-catch:** As excepcións comprobadas requiren ser incluídas nas declaracións *throws* dos métodos ou ser envoltas en bloques *try-catch*.
- ✓ **Non son subtipos de *RuntimeException* ou *Error*:** As excepcións comprobadas non son subtipos de *RuntimeException* ou *Error*. Derivan de [*Exception*](#) ou dunha subclase directa desta.
- ✓ **Forzan a xestión explícita:** O compilador obriga ao programador a manexar ou propagar excepcións comprobadas, contribuíndo a un manexo máis rigoroso de posibles erros.

4.1.2 Exemplos de excepcións comprobadas

Analizamos varios tipos de excepcións comprobadas e a súa utilidade:

- ✓ ***IOException*:** Ocorre cando hai problemas de entrada/saída, como non ser capaz de ler ou escribir en arquivos.

```
try {  
    FileReader arquivo = new FileReader("non_existente.txt"); // Pode  
    lanzar IOException  
} catch (IOException e) {  
    // Xestionar a excepción de entrada/saída  
}
```




- ✓ **SQLException:** Ocorre cando se traballa con bases de datos e hai erros nas operacións SQL (veremos isto na unidade 9).

```
try {  
    Connection con = DriverManager.getConnection("url_da_BD");  
    // Operacións SQL que poden lanzar SQLException  
} catch (SQLException e) {  
    // Xestionar a excepción de SQL  
}
```

- ✓ **FileNotFoundException:** Ocorre cando se intenta abrir un ficheiro que non existe.

```
try {  
    FileInputStream fis = new FileInputStream("ficheiro.txt"); // Pode  
    lanzar FileNotFoundException  
} catch (FileNotFoundException e) {  
    // Xestionar a excepción de ficheiro non encontrado  
}
```

4.1.3 Boas prácticas no uso de excepcións comprobadas

Manexar excepcións comprobadas correctamente é moi importante para ter unha aplicación robusta, fiable e cun seguimento correcto da súa execución. A continuación enumeramos unha serie de recomendacións a ter en conta:

- ➔ **Manexar ou propagar:** É esencial manexar ou propagar excepcións comprobadas. Se non se manexan localmente, deben ser propagadas utilizando a palabra chave `throws` na sinatura do método.
- ➔ **Xestión detallada:** Proporcionar mensaxes de erro informativas e detalladas axuda na identificación e resolución de problemas.
- ➔ **Evitar o uso de catch "vagos":** Evitar, na medida que sexa posible, o uso de catch vagos, onde se capture [*Exception*](#) de xeito xenérico sen especificar o tipo. Evitar a perda de información sobre o tipo específico de excepción, permitíranos ter máis información sobre o que acontece na nosa aplicación.
- ➔ **Manexo eficiente:** O manexo eficiente de excepcións comprobadas contribúe á robustez e fiabilidade do código.

As excepcións comprobadas proporcionan un control estrito sobre a xestión de erros no código, forzando ao programador a tomar acción explícita ante posibles problemas. Este control contribúe a un código máis fiable e a un manexo máis axeitado de situacións excepcionais.

4.2 Excepcións non comprobadas (Unchecked Exceptions)

As excepcións non comprobadas en Java son aquelas que non requiren que o programador as xestione de maneira explícita mediante cláusulas try-catch ou declarándoas na sinatura do método. Son excepcións que ocorren durante a execución dun programa e non se forza ao programador a manexalas, permitíndose propagar sen ser capturadas. Estas excepcións derivan de *RuntimeException* ou *Error*, e non requiren que se xestionen de maneira obrigatoria.

4.2.1 Características principais

A continuación, exploraremos as características e algúns exemplos de excepcións non comprobadas:

- ✓ **Non Requiren *throws* ou *try-catch*:** As excepcións non comprobadas non requiren ser incluídas nas declaracións throws dos métodos ou ser envoltas en bloques try-catch.
- ✓ **Subtipos de *RuntimeException* e *Error*:** As excepcións non comprobadas son subtipos de *RuntimeException* ou *Error*. Isto inclúe excepcións como *ArithmeticException*, *NullPointerException*, *ArrayIndexOutOfBoundsException*, entre outras, que derivan destes dous tipos de excepcións. Na vindeira unidade veremos como estender unha clase usando a herdanza.
- ✓ **Non son forzadas a ser capturadas:** Non é obrigatoria a captura ou xestión explícita destas excepcións. Iso pode levar a que erros non previstos interrompan a execución do programa, algo que coas excepcións comprobadas non ocorre.

4.2.2 Exemplos de excepcións non comprobadas

A continuación enumeramos varios tipos de excepcións non comprobadas e a súa utilidade:

- ✓ ***NullPointerException*:** Ocorre cando se intenta acceder a un obxecto que non foi inicializado (ten o valor null).

```
String texto = null;  
int lonxitude = texto.length(); // NullPointerException
```

- ✓ ***ArrayIndexOutOfBoundsException*:** Ocorre cando se intenta acceder a un índice que está fóra dos límites dun array.

```
int[] numeros = {1, 2, 3};  
int elemento = numeros[5]; // ArrayIndexOutOfBoundsException
```



- ✓ **ArithmeticException:** Ocorre cando se intenta realizar unha operación aritmética inválida, como a división por cero.

```
int resultado = 10 / 0; // ArithmeticException
```

- ✓ **IllegalArgumentException:** Ocorre cando se pasa un argumento non válido a un método.

```
public void metodo(int valor) {  
    if (valor < 0) {  
        throw new IllegalArgumentException("0 valor debe ser non  
negativo");  
    }  
    // Resto do código  
}
```

4.2.3 Boas prácticas no uso de excepcións non comprobadas

A pesar de que non son forzadas a ser capturadas, é importante manexar excepcións non comprobadas cando sexa posible. A continuación enumeramos unha serie de recomendacións a ter en conta:

- ➔ **Manexar excepcións relevantes:** Aínda que non sexan obrigatorias de capturar, é bo manexar excepcións non comprobadas que sexan críticas para a estabilidade do programa.
- ➔ **Prover información de erros:** Cando se lanza unha excepción non comprobada, prover información detallada sobre o erro pode axudar na súa resolución.
- ➔ **Validación de datos:** Realizar validación de datos antes de operacións críticas pode axudar a evitar excepcións non desexadas.
- ➔ **Logs e rexistro de erros:** Usar rexistros de erros (logs) para gardar información sobre excepcións non comprobadas pode ser crucial para a depuración e mellora continua do software (isto verémolo na unidade 8).

Podemos concluír que a comprensión e o manexo adecuado de excepcións non comprobadas contribúen a un código máis robusto e fiable. Aínda que non sexan forzadas a ser capturadas, considerar a súa xestión contribúe á estabilidade do programa e á mellora da experiencia do usuario.

5 Propagación de excepcións: Instrución throws

Un método pode propagar unha excepción utilizando a palabra clave `throws` na súa declaración. A propagación de excepcións é un concepto fundamental en Java que se refire á transmisión dunha excepción dun método a outro ao longo da cadea de chamadas de métodos. Cando ocorre un erro nun método e este non é capaz de manexar a excepción localmente, a excepción propagarase cara arriba ata que se atope un bloque `catch` que poida manexala ou ata que alcance o nivel máis alto do programa, onde o programa pode finalizar con información sobre o erro.

Imaxinemos un escenario onde temos varios métodos chamándose uns aos outros e unha excepción ocorre nun nivel inferior. A propagación de excepcións permite que a información sobre a excepción se transporte a través dos distintos niveis da pila de chamadas ata que se atope un punto onde se poida manexar adecuadamente.

Vexamos un exemplo para ilustrar a propagación de excepcións:

```
public class ExemploPropagacion {

    public static void main(String[] args) {
        try {
            metodoExterno();
        } catch ( Exception e) {
            System.out.println("Excepción capturada: " + e.getMessage());
        }
    }

    public static void metodoExterno() throws Exception {
        try {
            metodoIntermedio();
        } catch ( Exception e) {
            throw e; // Propaga a excepción cara arriba
        }
    }

    private static void metodoIntermedio() throws Exception {
        metodoInterno();
    }

    private static void metodoInterno() throws Exception {
        // Simulación dun erro que xera unha excepción personalizada
        throw new Exception("Ocorreu un erro no método interno");
    }

}
```



Neste exemplo, temos catro métodos estáticos (da clase): `main`, `metodoExterno`, `metodoIntermedio` e `metodoInterno`, que se chaman uns aos outros do xeito indicado. Cando `metodoInterno` xera unha excepción, esta excepción propágase cara arriba a través dos métodos ata que se atopa un bloque *catch* que pode manexala. Neste caso, o bloque *catch* está no método `main`, onde se imprime unha mensaxe indicando que se capturou a excepción.

A propagación de excepcións permite unha xestión flexible de erros, onde os métodos poden decidir se manexan localmente as excepcións ou se as propagan cara arriba para que se traten noutro lugar. Lembre sempre especificar as excepcións que un método pode lanzar utilizando a declaración *throws* na súa firma.